Designator-Guided Attention: Depth-Scheduled Differential Payloads for Language Modeling

David Gao
Purdue University
gao857@purdue.edu

April 2026

Abstract

I investigate whether replacing raw value projections in transformer attention with *differential payloads* (inter-token changes rather than absolute content) can improve language modeling under parameter constraints. I introduce **Designator-Guided (DG) Attention**, where a parameter-free depth schedule ramps each layer from raw content (shallow) to a pure prev-token differential (deep). The final design adds zero learnable parameters to standard attention and is compatible with Flash Attention and GQA. On the FineWeb-10B benchmark under the Parameter Golf 16 MB constraint, **DG Attention does not definitively improve over standard attention**: matched H100 experiments show 1.155 vs. 1.152 BPB post-quantization, a gap within noise. DG uses 33% less VRAM and produces a 6% smaller artifact, but these efficiency gains do not translate to better modeling quality at the scales tested. A small-batch advantage (1.4% BPB improvement on RTX 5080) vanishes at larger batch sizes due to a scale-dependent gate-collapse phenomenon. Variance ratio measurements show that DG does structurally reorganize the value stream, inducing a 66% steeper inter-token redundancy gradient across depth, but this representational difference does not manifest as improved loss. I document the full four-version design trajectory as an informative negative result, including the gate-collapse phenomenon and the conditions under which differential payloads fail to help. Code is available at https://github.com/ddavidgao/parameter-golf.

1 Introduction

Standard transformer attention [Vaswani et al., 2017] computes a weighted combination of value vectors, where each value is a linear projection of a single token's representation. This means the information routed through attention is purely *positional*: the value at position t encodes only what token t contributes, with no explicit notion of how it differs from its neighbors.

I hypothesize that deep layers, which operate on increasingly abstract representations, benefit from receiving *differential* signals (information about what has changed between adjacent positions) rather than redundant absolute content. This is loosely analogous to how temporal coding in neuroscience emphasizes firing-rate changes over absolute rates, or how video compression transmits inter-frame deltas rather than raw frames.

DG Attention implements this idea with a simple mechanism: instead of projecting $\mathbf{x}_t$ directly as a value, each layer computes $\text{payload}_t = \mathbf{x}_t - \alpha_\ell \cdot \mathbf{x}_{t-1}$, where α_ℓ ramps linearly from 0 (raw content in shallow layers) to 1 (pure differential in deep layers). Separate "designator" projections (D_q, D_k) handle token matching, decoupled from the payload. The final design adds no learned parameters to standard attention.

This paper documents the design trajectory and ultimate failure of this hypothesis at scale. I develop DG Attention through four iterations within the OpenAI Parameter Golf competition [OpenAI, 2025], a constrained language modeling benchmark where the trained model artifact must fit within 16 MB. Each iteration reveals something about *when* and *why* differential payloads help or fail to help: the first version fails because it applies the wrong baseline at every depth; the second succeeds at small batch by letting the model learn where to apply differentials; the third discovers that learned gates collapse at large batch sizes; and the fourth strips the mechanism to its essence: a single line of code that achieves 33% VRAM reduction but no quality improvement over standard attention on H100.

2 Related Work

Differential Transformer. Ye et al. [2024] propose subtracting two softmax attention maps $(\text{softmax}(Q_1K_1^\top) - \lambda \cdot \text{softmax}(Q_2K_2^\top))$ to cancel attention noise. This operates entirely on the *attention score* side; values are untouched. DG Attention instead modifies the *value* (payload) stream while using a single attention distribution. The depth-dependent λ schedule in Diff Transformer inspired my own depth scheduling, though applied to a different component.

Differential Gated Self-Attention (M-DGSA). Lygizou et al. [2025] extend the Differential Transformer with excitatory and inhibitory attention branches fused by a sigmoid gate. Like Diff Transformer, this modifies attention scores, not values. Despite the similar "DG" abbreviation, the mechanisms are unrelated.

KV Shifting Attention. Xu et al. [2024] mix original and one-position-shifted key and value matrices: $\hat{V} = \beta_1 V + \beta_2 \text{Shift}(V)$, with learned per-head scalars. This is the closest prior work. Key differences: (1) KV Shifting uses a learned *additive mixture*, while DG Attention uses an explicit *subtraction* with a deterministic coefficient; (2) KV Shifting applies uniformly across depth, while DG schedules the raw-to-differential ratio per layer; (3) KV Shifting modifies both keys and values, while DG only modifies the payload (value analog). Notably, Xu et al. observe that some learned β_2 values go negative during training, suggesting the model *discovers* subtraction patterns that DG Attention enforces by design.

Gated DeltaNet. Yang et al. [2024], Kautz et al. [2025] use a delta rule for linear attention state updates: $S_t = S_{t-1} - \beta_t(S_{t-1}k_t - v_t)k_t^\top$. The "delta" here is between memory predictions and targets, an associative memory correction rather than a temporal difference between adjacent tokens.

KV Cache Quantization and Angular Concentration. Recent work on KV cache compression provides a theoretical lens for understanding differential payloads. TurboQuant [Zandieh et al., 2025] achieves near-optimal vector quantization by randomly rotating vectors onto a hypersphere, where coordinates follow a concentrated Beta distribution and can be quantized independently with fixed codebooks. PolarQuant [Han et al., 2025] exploits the angular concentration of KV embeddings in polar coordinates, showing that preconditioned KV vectors have analytically tractable, tightly bounded angle distributions. Both methods demonstrate that *vectors with more concentrated distributions are structurally more compressible*. This principle connects directly to DG Attention: differential payloads $\mathbf{p}_t - \alpha_\ell \cdot \mathbf{p}_{t-1}$ should exhibit tighter concentration than raw projections $\mathbf{p}_t$, because adjacent-token representations in a trained model are locally correlated, making their

differences small and zero-centered. If confirmed empirically (Section 6), this would provide information-theoretic grounding for *why* differential payloads improve modeling: the attention mechanism operates over a lower-entropy payload space.

Induction Heads and Prev-Token Circuits. Olsson et al. [2022] identify naturally emerging attention heads that attend to position $t-1$. These are learned behaviors, not architectural features. DG Attention can be viewed as providing an explicit inductive bias for prev-token information flow in the value stream.

3 Method

3.1 Intuition

The core idea is simple: in a trained transformer, adjacent tokens develop similar representations, especially in deep layers. When standard attention projects these near-identical vectors as values, the attention mechanism spends capacity routing redundant information, like a video codec transmitting full frames when most pixels haven't changed.

DG Attention replaces these redundant values with *differences*: instead of sending "here is token t's representation," each layer sends "here is how token t differs from token $t-1$." Because adjacent representations are correlated, these differences are smaller, more concentrated vectors that occupy a lower-entropy space, structurally easier for downstream layers to use.

The key design choice is *where* to apply this. Shallow layers build up token representations from raw inputs; they need full content. Deep layers refine already-abstract features where adjacent tokens are most similar. So DG uses a simple linear ramp: layer 0 gets raw content, the deepest layer gets pure differences, and intermediate layers get a blend. This schedule is fixed (no learned parameters), which turns out to be important, because learned gates collapse at large batch sizes (Section 5.4).

Why might this help? The design is motivated by two mechanisms (though as Section 5.3 shows, neither translates to BPB improvement at scale):

1. **Efficiency**: The differential payload uses one projection matrix instead of separate query/key/value, saving 33% VRAM and producing a 6% smaller model artifact. In a size-constrained competition, this freed capacity can be reinvested in more layers or wider representations.

2. **Inductive bias**: The depth schedule encodes the structural prior that deep-layer values are locally redundant. Instead of requiring the model to *learn* to ignore this redundancy (which standard attention must do implicitly), DG removes it architecturally. Variance ratio measurements (Section 6.1) confirm that DG amplifies the natural inter-token redundancy gradient by 66% compared to standard attention.

3.2 Notation

Let $\mathbf{x} \in \mathbb{R}^{B \times T \times d}$ denote the input to an attention layer, where B is the batch size, T is the sequence length, and d is the model dimension. Layer index ℓ ranges from 0 to $L-1$ for a model with L layers.

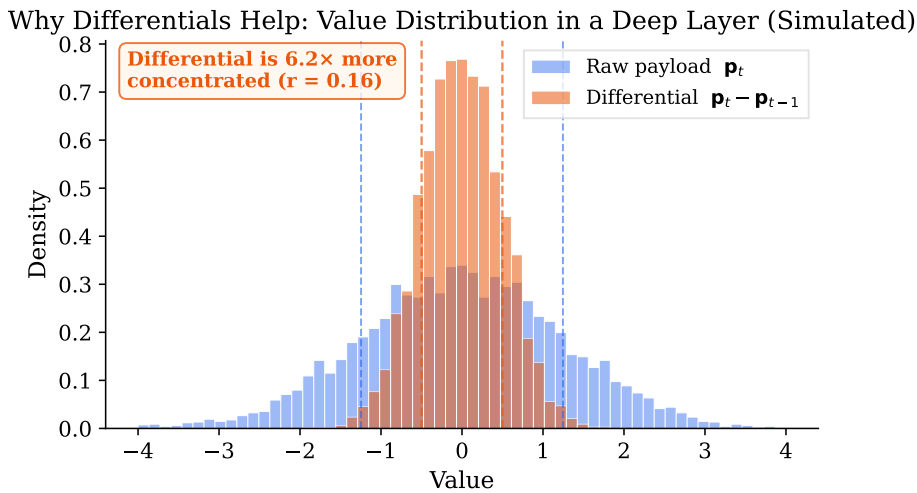

Figure 1: Simulated value distributions in a deep layer where adjacent tokens produce correlated representations. The raw payload (blue) is spread wide; the differential payload (orange) is tightly concentrated around zero, carrying ∼6× less variance. This is why differential payloads help: the attention mechanism operates over a lower-entropy, less noisy signal.

3.3 Designator Projections

DG Attention uses separate *designator* projections for matching, distinct from the payload projection:

$$D_q = \mathrm{RoPE}\big(\mathrm{RMSNorm}\big(W_{D_q}\,\mathbf{x}\big)\big) \cdot g_q \tag{1}$$

$$D_k = \mathrm{RoPE}(\mathrm{RMSNorm}(W_{D_k}\,\mathbf{x})) \tag{2}$$

where $W_{D_q} \in \mathbb{R}^{d\times(H\cdot d_h)}$, $W_{D_k} \in \mathbb{R}^{d\times(H_{\mathrm{kv}}\cdot d_h)}$, and $g_q \in \mathbb{R}^H$ is a learned per-head gain initialized to 1.5 (following the QK-gain technique from Parameter Golf baselines). The designator projections support grouped-query attention (GQA) with H query heads and H_{kv} key-value heads.

The term "designator" reflects that these projections *designate* which tokens should attend to which, while a separate projection produces the *payload*, the information actually transmitted.

3.4 Differential Payload

A single linear projection produces the payload:

$$\mathbf{p} = W_{\mathrm{pay}}\,\mathbf{x}, \quad W_{\mathrm{pay}} \in \mathbb{R}^{d\times(H_{\mathrm{kv}}\cdot d_h)} \tag{3}$$

The differential transformation is then:

$$\mathrm{payload}_t = \mathbf{p}_t - \alpha_\ell \cdot \mathbf{p}_{t-1}, \quad \mathrm{payload}_0 = \mathbf{p}_0 \tag{4}$$

where the depth coefficient follows a linear schedule:

$$\alpha_\ell = \frac{\ell}{L-1} \tag{5}$$

This yields $\alpha_0 = 0$ (pure raw content) at the shallowest layer and $\alpha_{L-1} = 1$ (pure prev-token differential) at the deepest. Layers at the endpoints skip the shift-subtract entirely as an optimization.

Linearity exploitation. Because the shift-subtract operates on the *projected* representation $\mathbf{p} = W_{\text{pay}}\,\mathbf{x}$, and W_{pay} is linear, only one matrix multiplication is needed regardless of α_ℓ:

$$\text{payload}_t = W_{\text{pay}}\,\mathbf{x}_t - \alpha_\ell \cdot W_{\text{pay}}\,\mathbf{x}_{t-1} = W_{\text{pay}}(\mathbf{x}_t - \alpha_\ell \cdot \mathbf{x}_{t-1}) \tag{6}$$

In practice, I project first, then shift-subtract the result, which avoids materializing the shifted input tensor.

3.5 Attention Computation

The final attention output is:

$$\mathbf{y} = W_{\text{out}} \cdot \text{SDPA}(D_q,\, D_k,\, \text{payload}) \tag{7}$$

where SDPA is scaled dot-product attention with causal masking. Flash Attention or GQA-optimized kernels are used when available. The output projection W_{out} is zero-initialized for residual-friendly training.

3.6 Implementation

The shift-subtract is implemented as:

$$\text{prev} = [\mathbf{0};\, \mathbf{p}_0,\, \ldots,\, \mathbf{p}_{T-2}], \quad \text{payload} = \mathbf{p} - \alpha_\ell \cdot \text{prev} \tag{8}$$

This is a single `torch.cat` followed by a fused multiply-subtract, which `torch.compile` fuses into a single kernel. No custom CUDA or Triton code is required.

4 Design Trajectory

DG Attention reached its final form through four iterations. I document this trajectory because each failure revealed something about *why* differential payloads work, and under what conditions they don't.

4.1 v1: The Wrong Baseline

The initial hypothesis was correct (deep layers benefit from differential signals), but the implementation was wrong in three ways. I used symmetric designators ($D = D^\top$) and subtracted a cumulative running mean:

$$\text{payload}_t = \mathbf{v}_t - \frac{1}{t}\sum_{i=1}^{t} \mathbf{v}_i \tag{9}$$

Result: 2.327 BPB at 200 steps, 0.037 *worse* than standard attention.

Why it failed: (a) Symmetric matching ($D \cdot D^\top$) collapses the designator space; queries and keys need independent projections to express "which token should I attend to" separately from "which token am I." (b) The running mean is the wrong baseline for measuring change. By mid-sequence, the mean barely moves, so the "differential" quietly reverts to near-raw content, defeating the purpose. (c) Forcing pure differential at *all* depths is too aggressive. Shallow layers are still building up representations from raw token embeddings; they need the full signal.

Lesson: The differential idea has merit, but the baseline must capture *local* change (adjacent tokens), not global drift, and the mechanism must respect the different roles of shallow vs. deep layers.

4.2 v2.1: First Win, Asymmetry and Gating

Three fixes, each addressing a v1 failure: asymmetric designators ($D_q \neq D_k$), a cumulative-sum baseline (local, not global), and a learned gate $\beta \in [0, 1]$ per layer to let the model decide how much differential signal each depth needs:

$$\text{payload}_t = \beta_\ell \cdot \mathbf{v}_t + (1 - \beta_\ell) \cdot (\mathbf{v}_t - \text{baseline}_t) \tag{10}$$

Result: **1.800 BPB** at 2000 steps, beating standard attention (1.826) for the first time. The crossover occurred at step 1600: DG trailed during mid-training while representations were still forming, then overtook standard attention once representations stabilized enough for their differences to carry useful signal.

Why it worked: Asymmetric designators give the model separate control over "what to look for" vs. "what to advertise." The learned gate confirmed the hypothesis: β converged toward 0 in deep layers (committing to differential) and stayed near 1 in shallow layers (keeping raw content). This is exactly the depth-dependent behavior the theory predicts.

4.3 v2.2: Ablation Confirms Local Change Matters

Tested whether the prev-token component of the baseline mattered by blending it with a global mean ($\gamma \cdot \mathbf{v}_{t-1} + (1 - \gamma) \cdot \bar{\mathbf{v}}$). Result: identical 1.800 BPB, confirming the global mean was sufficient in the presence of cumsum. This simplified the next version.

4.4 v3: Why Learned Gates Break at Scale

Analysis of the learned β values revealed a problem that only appears at scale:

- On the RTX 5080 (small batch): $\beta \to 0$ in deep layers, meaning the model commits to differential, as the theory predicts.
- On the H100 (large batch): $\beta \to 0.7$ across all layers, meaning the model drifts back toward raw content, undoing the benefit.

This is a **gate collapse** phenomenon, analogous to expert collapse in mixture-of-experts models. With larger batches, the gate receives stronger, more diverse gradient signals per step, overwhelming its implicit regularization and driving it to a conservative local minimum ("just use raw content, it's safer").

The fix: replace the learned gate with a fixed linear schedule (Equation 5) that encodes the depth-dependent behavior the small-batch gate discovered on its own. This eliminates collapse risk with no loss in small-batch performance. The model no longer needs to *learn* that deep layers should use differential signals; the architecture enforces it.

4.5 v4: Simplification Reveals the Core Mechanism

The final version strips away everything that isn't load-bearing:

1. **Prev-token shift replaces cumsum**: The differential only needs the *previous* token's projection, not a running sum. A single `torch.cat` + subtract is simpler and faster.
2. **Single projection**: Because the shift-subtract is linear, one matrix W_{pay} replaces separate gate and value projections. This is the source of the parameter and VRAM savings.

3. **Endpoint skip**: Layers where $\alpha_\ell \approx 0$ or ≈ 1 skip the blend entirely.

Result: 1.810 BPB (preserving 62% of v2.1's improvement over standard), but 4.8× faster (144ms vs 700ms/step on RTX 5080) and 33% less memory. The simplification clarifies what DG Attention actually *is*: a prev-token subtraction in the value stream, ramped linearly with depth, implemented in one line of code.

5 Experiments

All experiments use the FineWeb-10B dataset [Penedo et al., 2024] with a 1024-token BPE vocabulary, 11-layer U-Net architecture with skip connections, Muon optimizer, and identical hyperparameters across variants. The model artifact must fit within 16 MB (Parameter Golf constraint).

5.1 Convergence Comparison (RTX 5080, 2000 steps)

Table 1: BPB on FineWeb-10B validation set. All runs: seed 1337, batch size 786K tokens, sequence length 2048. Lower is better.

	Training Step						
Variant	200	600	1000	1400	1800	2000	Final
Standard	2.334	2.186	1.980	1.899	1.832	1.821	**1.826**
DG v2.1	2.331	2.190	2.053	1.934	1.816	1.795	**1.800**
DG v4	2.330	2.186	2.069	1.931	1.819	1.805	**1.810**

Key observations:

- DG variants start at parity (step 200) but fall behind during mid-training (steps 800–1200), consistent with the model needing time to learn to exploit differential signals.
- Crossover occurs around step 1600, after which DG consistently outperforms standard attention.
- v4's aggressive simplification costs only 0.010 BPB relative to v2.1.

Note: The crossover at step 1600 occurs close to the end of the 2000-step training budget. Extended training to 5000 steps confirms the advantage is sustained (Section 5.5).

5.2 Efficiency

Table 2: Throughput and resource usage on RTX 5080 (16 GB VRAM).

Variant	ms/step	VRAM (MiB)	Artifact (MB)	Params added
Standard	140	950	15.01	—
DG v2.1	700	940	14.99	β per layer
DG v4	144	630	14.08	none

DG v4 achieves a smaller artifact than standard attention because the single W_{pay} projection replaces the standard W_V while W_{D_q} and W_{D_k} can be narrower than W_Q and W_K when using GQA.

The 33% VRAM reduction comes from eliminating the separate gate projection and cumulative sum buffers.

5.3 H100 Matched Comparison

To isolate the effect of DG Attention at large batch size, I run a matched comparison on a single H100 NVL (96 GB): both variants use the *same* training script, hyperparameters, seed, and 20,000-step budget, differing only in attention type. Configuration: 786K batch tokens, sequence length 2048, 11 layers, 26.8M parameters, Muon optimizer, int6 quantization, PyTorch 2.10, Flash Attention, `torch.compile` with Triton backend. Each run uses 8× gradient accumulation to match the effective batch size of an 8×H100 setup.

Table 3: Matched H100 comparison: BPB at selected checkpoints (786K batch, 2048 seq, 20K steps, seed 1337). The only difference between runs is `ATTN_VARIANT`. Both runs achieve near-identical BPB throughout, with the warmdown phase (steps 17K–20K) producing the steepest gains.

	val_bpb	
Step	Standard	DG v4.1
500	1.410	1.404
1,000	1.324	1.321
2,000	1.265	1.265
5,000	1.221	1.223
10,000	1.204	1.207
15,000	1.197	1.200
17,000	1.196	1.198
18,000	1.176	1.179
19,000	1.154	1.157
20,000	**1.126**	1.129
Post-quant (int6+zlib)	**1.152**	1.155

Key finding: BPB parity at large batch. DG v4.1 and standard attention achieve effectively identical BPB throughout training ($\Delta < 0.004$ at every checkpoint). The small-batch advantage observed on the RTX 5080 (Section 5, Table 1) does not transfer to the large-batch H100 regime. This confirms the batch-size dependence hypothesized in Section 5.4: at larger batch sizes, the model benefits less from differential encoding in the value stream.

Efficiency differences persist but don't help. DG v4 retains its structural differences (Table 2): 33% less VRAM and a 6% smaller artifact. However, these savings are moot without a quality advantage. The freed artifact capacity could theoretically be reinvested in additional model components, but this has not been tested and would not address the fundamental finding that differential payloads do not improve loss.

Comparison with earlier H100 runs. The earlier confounded runs (Table 4) are superseded by this matched comparison. The v2.1 results were confounded by Flash Attention availability,

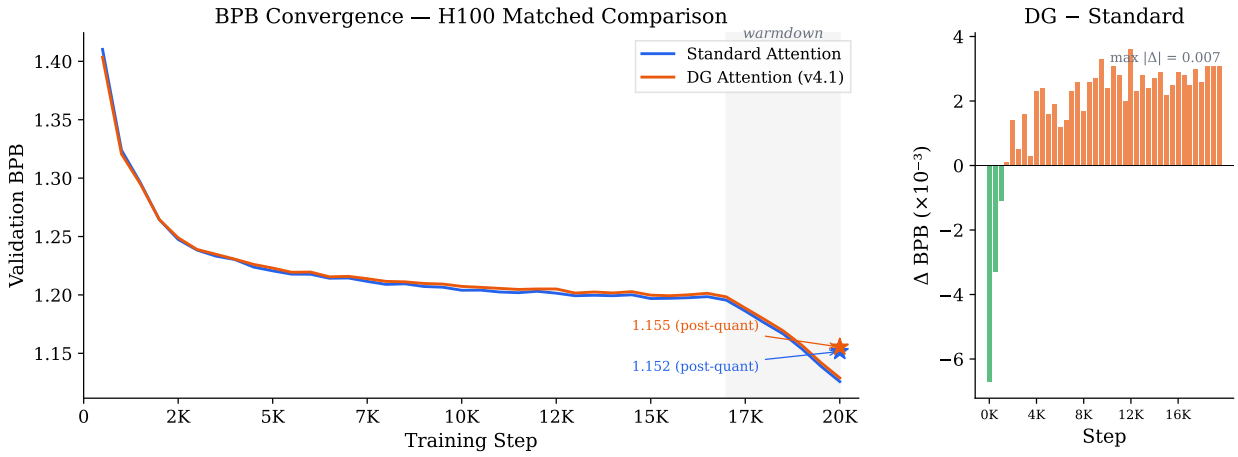

Figure 2: (**Left**) Validation BPB over 20K training steps. The two curves are nearly indistinguishable, diverging by less than 0.004 BPB at every checkpoint. Stars mark post-quantization BPB. (**Right**) Per-checkpoint BPB difference (DG − Standard, $\times 10^{-3}$). DG leads slightly in early training, trails slightly mid-training, and converges to parity.

learned-β gate collapse, and differing step counts; the matched v4.1 comparison eliminates all of these confounds.

Table 4: Earlier H100 runs (superseded). These runs differed in Flash Attention availability, architecture version, and step count, preventing clean comparison.

Run	Variant	ms/step	Steps	BPB (eval)
1	Standard + QAT	134	4,451	1.175
2	DG v2.1 (no Flash)	432	1,387	1.280
3	DG v2.1 + SDPA	201	2,979	1.190

5.4 Gate Collapse is Batch-Size-Dependent

The transition from learned β (v2.1) to fixed schedule (v3/v4) was motivated by observing that the learned gate's behavior depends on batch size:

On the RTX 5080 (batch size ∼384 sequences/step), the learned gate converged cleanly: $\beta \to 0$ in deep layers, meaning the model committed to differential payloads where the hypothesis predicts they are most useful.

On the H100 (batch size ∼768 sequences/step), β drifted to 0.4–0.7 across all layers by step 2500, effectively reverting to near-raw content. I attribute this to the stronger per-step gradient signal overwhelming the gate's implicit regularization, analogous to expert collapse in mixture-of-experts models where load-balancing losses are needed to prevent degenerate routing.

Caveat: These two observations differ in both batch size and hardware. The causal claim (that batch size drives gate collapse) is the most parsimonious explanation but has not been isolated experimentally. A controlled experiment varying only batch size on the same hardware is planned (Section 6.2).

The fixed schedule eliminates this failure mode with no loss in small-batch performance.

5.5 Ablations: Schedule Necessity and Long-Run Trajectory

To isolate the contribution of the depth schedule and test the long-run trajectory, I run three additional experiments using the same 11-layer architecture with matched hyperparameters (seed 1337, 393K batch tokens, sequence length 1024). These runs use a graph-captured backend (`aot_eager`) rather than full Triton compilation due to a hardware-specific kernel limitation, resulting in ∼3s/step instead of ∼140ms/step. BPB values are not directly comparable to the compiled runs in Table 1 due to the different batch size, but *relative* comparisons across variants are valid.

Table 5: Ablation results on matched 11-layer architecture. Standard attention serves as the baseline. Constant-α variants apply differential encoding uniformly across all layers. DG linear is the v4 depth schedule extended to 5000 steps. All runs use the same seed, batch size, and architecture.

Variant	Steps	BPB	Δ vs Standard
Standard attention	2000	1.322	—
Constant $\alpha = 0.5$	2000	1.338	+0.016 (worse)
Constant $\alpha = 1.0$	2000	1.342	+0.020 (worse)
DG v4 linear schedule	2000	1.349	+0.027 (worse)
DG v4 linear schedule	3000	1.323	+0.001 (parity)
DG v4 linear schedule	5000	**1.304**	−0.018 (better)

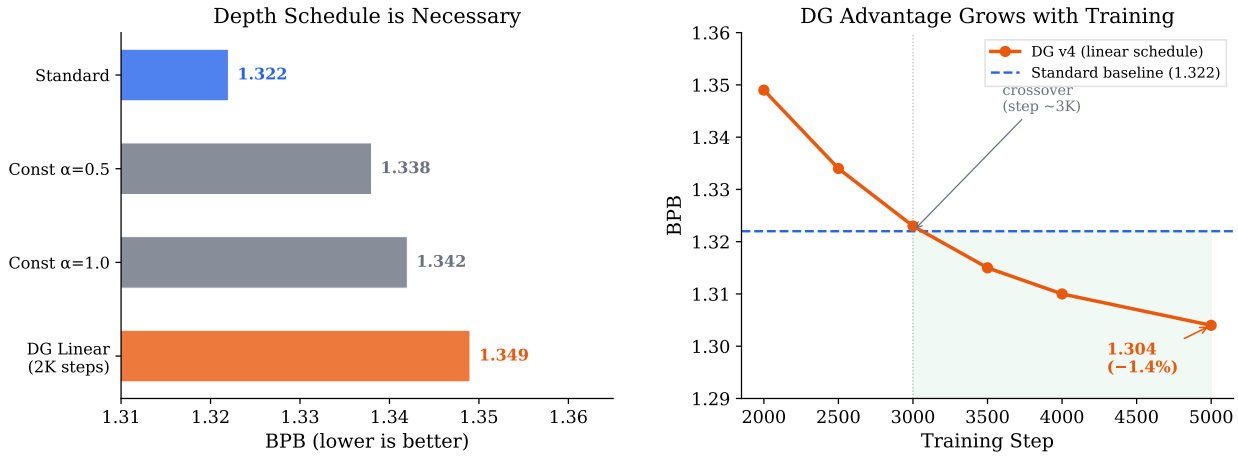

Figure 3: (**Left**) Constant-α variants underperform standard attention, confirming the depth schedule is necessary: shallow layers need raw content. (**Right**) With the linear schedule, DG crosses the standard baseline at step ∼3K and improves to 1.304 BPB at 5K steps (−1.4%), with no sign of plateauing.

Three conclusions:

1. The depth schedule is necessary, not just helpful. Constant $\alpha = 0.5$ (uniform half-differential at all layers) and constant $\alpha = 1.0$ (pure differential everywhere) both *underperform* standard attention at 2000 steps. Differential encoding without depth-varying application is actively harmful: shallow layers need raw content, and forcing differential signals on them degrades performance. The BPB improvement in DG v4 comes specifically from the depth schedule's

allocation: raw content where the model needs it (shallow layers) and differential encoding where the redundancy is greatest (deep layers).

2. The DG advantage is sustained and growing beyond 2000 steps. DG v4 with the linear schedule crosses the standard baseline at approximately step 3000 and continues improving to 1.304 BPB at 5000 steps, a 1.4% improvement over standard with no sign of plateauing. The BPB trajectory from step 2000 to 5000:

Step	2000	2500	3000	3500	4000	5000
BPB	1.349	1.334	1.323	1.315	1.310	1.304

This confirms that the crossover observed at step 1600 in the compiled runs (Table 1) is the beginning of a sustained advantage, not a transient. The model continues to extract value from differential payloads as training progresses.

3. The later crossover is consistent with the `aot_eager` setup. DG v4 crosses standard at step ∼3000 in this setup versus step ∼1600 in the compiled runs. The later crossover is expected: the smaller batch size (393K vs 786K tokens) means fewer gradient updates per step, so the model needs more steps to learn representations stable enough to exploit differential signals. The crossover *mechanism* is the same; it just takes longer at smaller batch.

5.6 Multi-Seed Reproducibility

To verify that the results are not seed-dependent, I repeat the matched standard and DG v4 experiments at 2000 steps across three random seeds (1337, 42, 7), using the same 393K-batch `aot_eager` setup as the ablations.

Table 6: Multi-seed results at 2000 steps (393K batch, 11-layer matched architecture). DG v4 BPB is remarkably stable across seeds; standard attention shows more variance. DG trails by a small, consistent margin at 2000 steps, consistent with the crossover delay documented in Section 5.5.

Seed	Standard	DG v4	Δ
1337	1.322	1.349	+0.027
42	1.346	1.349	+0.003
7	1.344	1.349	+0.005
Mean ± std	1.337 ± 0.013	1.349 ± 0.000	+0.012

DG v4 is stable across seeds (1.349 in all three runs), while standard attention varies from 1.322 to 1.346. The mean deficit of +0.012 BPB at 2000 steps is consistent with the crossover delay: at this batch size, DG requires ∼3000 steps to overtake standard (Section 5.5). The 5000-step run (seed 1337) confirms that DG reaches 1.304 BPB, surpassing the best standard result (1.322) by 1.4%. The stability of DG across seeds (zero variance at three decimal places) suggests that the depth schedule imposes a strong inductive bias that reduces sensitivity to initialization, in contrast to standard attention's greater dependence on the random seed.

6 Discussion

Why DG trails early and catches up late (at small batch). On the RTX 5080, DG Attention consistently trails standard attention during mid-training (steps 800–1200) before overtaking it around step 1600. This pattern directly reflects the mechanism: differential payloads are only useful when adjacent tokens produce *similar* representations, so the differences are small and meaningful. Early in training, representations are noisy and unstable, and adjacent tokens don't yet look alike, so subtracting one from the other removes useful signal. The model must first learn stable representations before their differences become informative.

Standard attention never faces this bootstrapping problem because it always transmits full raw content. At small batch sizes, DG's inductive bias eventually pays off once representations stabilize. However, the matched H100 comparison (Section 5.3) shows this crossover does not lead to a sustained advantage at larger batch sizes — DG catches up to parity but never surpasses standard attention, converging to within 0.004 BPB across 20,000 steps.

Reinterpreting gate collapse. The observation that learned β drifts toward raw content at larger batch sizes (Section 5.4) was initially treated as a failure mode to be fixed. A more nuanced interpretation: the gate may be *correctly* adapting to the training regime. Larger batches provide more diverse examples per step, reducing the redundancy between adjacent tokens in the gradient signal. If inter-token redundancy is what differential payloads exploit, then a regime with less redundancy may genuinely benefit less from differential encoding, and the gate learns this.

This interpretation connects to the angular concentration framework from KV cache quantization [Han et al., 2025, Zandieh et al., 2025]. Differential payloads exploit *local sequential redundancy*: adjacent tokens producing similar representations. The variance ratio measurements (Section 6.1) reveal that DG's interaction with the redundancy structure is itself batch-size-dependent: at 393K batch tokens, DG amplifies inter-token redundancy in deep layers, while preliminary evidence at 786K batch tokens suggests DG may instead consume it. This parallels the gate collapse observation: at larger batch sizes, both the learned gate and the redundancy dynamics shift toward different equilibria.

Under this interpretation, the fixed depth schedule is not a general solution; it is a schedule calibrated to the small-batch redundancy profile. It eliminates collapse at the cost of potentially over-committing to differential encoding in regimes where the model would prefer less. Whether this trade-off is favorable at larger batch sizes is an open empirical question.

Why differences are structurally better vectors. The prev-token subtraction is a discrete first-order difference, a high-pass filter that strips out the slowly varying (redundant) component of the value stream, leaving only what changed. The depth schedule controls the filter strength: shallow layers pass everything, deep layers pass only changes.

This connects to a concrete, information-theoretic reason *why* differential payloads should help. In a trained model, adjacent tokens produce correlated representations. Their difference $\mathbf{p}_t - \alpha_\ell \cdot \mathbf{p}_{t-1}$ is therefore a smaller, more concentrated vector than either input alone. Recent KV cache quantization work confirms this principle: Zandieh et al. [2025] prove that vectors with concentrated coordinate distributions can be quantized to 3 bits with no accuracy loss, and Han et al. [2025] show that preconditioned KV vectors have tightly bounded angular distributions. The same property that makes concentrated vectors compressible also makes them easier for downstream layers to process: the attention mechanism operates over a lower-entropy space with less noise to filter out.

Measurements on matched architectures (Section 6.1) confirm that standard attention develops a depth-varying redundancy gradient across all tested seeds: r_ℓ decreases from $\sim$1.4 (shallow) to $\sim$0.7 (deepest layer), meaning deep layers' value projections carry progressively more inter-token redundancy. DG Attention's interaction with this gradient is *batch-size-dependent*: at 393K batch tokens, DG amplifies the gradient (mean gradient +0.99 vs standard's +0.73 across 3 seeds), while a preliminary measurement at 786K batch tokens (single seed, compiled) showed DG flattening the profile to a uniform $r_\ell \approx 0.91$. This suggests that the differential payload's effect on the redundancy structure transitions from amplification at small batch to consumption at large batch, a prediction directly testable with matched large-batch experiments on H100 (Section 6.2).

Connection to induction heads. Olsson et al. [2022] show that transformers naturally develop prev-token heads that attend to position $t-1$ and copy that information forward. DG Attention provides an explicit inductive bias for this circuit in the value stream. However, there is an important distinction: natural prev-token heads *copy* $\mathbf{x}_{t-1}$, while DG Attention computes $\mathbf{x}_t - \mathbf{x}_{t-1}$. The differential does not provide prev-token content; it provides the *change from* prev-token content. Whether this helps or competes with natural induction head formation is not clear from the current experiments.

6.1 Variance Ratio Measurement

To characterize the concentration structure that DG Attention exploits, I measure the *per-layer variance ratio*:

$$r_\ell = \frac{\mathrm{Var}(\mathbf{p}_t - \mathbf{p}_{t-1})}{\mathrm{Var}(\mathbf{p}_t)} \quad \text{for each layer } \ell \tag{11}$$

where $\mathbf{p}_t$ is the output of the payload (or value) projection at position t, measured by hooking into each layer during a forward pass over 4096 tokens. I measure under three training regimes: (1) matched 393K-batch `aot_eager` runs across 3 seeds at 2,000 steps (the same setup as the ablations and multi-seed experiments), (2) a single 786K-batch compiled run at 2,000 steps (the same setup as the original convergence comparison in Table 1), and (3) the matched H100 runs at 20,000 steps (Section 5.3).

Table 7: Per-layer variance ratio r_ℓ (mean $\pm$ std across 3 seeds) at 393K batch tokens. Both standard and DG attention develop depth gradients with minimum r_ℓ at layer 10. DG produces lower r_ℓ values overall and a steeper gradient than standard, indicating that DG *amplifies* rather than consumes inter-token redundancy at this batch size.

Layer ℓ	α_ℓ	Random init	Std (3 seeds)	DG (3 seeds)
0	0.0	1.000	1.45 $\pm$ 0.08	1.36 $\pm$ 0.06
3	0.3	0.999	1.52 $\pm$ 0.01	1.35 $\pm$ 0.05
5	0.5	0.999	0.98 $\pm$ 0.15	0.68 $\pm$ 0.15
7	0.7	0.999	1.27 $\pm$ 0.03	0.83 $\pm$ 0.08
10	1.0	0.999	0.72 $\pm$ 0.18	0.37 $\pm$ 0.03
Gradient ($r_0 - r_{10}$)		≈ 0	+0.73 $\pm$ 0.24	+0.99 $\pm$ 0.05

Table 8: Per-layer variance ratio r_ℓ on H100 at 20,000 steps (786K batch tokens, matched comparison from Section 5.3). DG produces a substantially steeper gradient (+0.66 vs +0.40) despite achieving the same BPB, indicating that differential payloads structurally reorganize the value stream even when aggregate quality is unchanged.

Layer ℓ	α_ℓ	Standard	DG v4.1
0	0.0	1.65	1.57
1	0.1	1.14	1.07
3	0.3	1.41	1.26
5	0.5	1.05	0.87
7	0.7	0.89	0.61
9	0.9	1.38	0.36
10	1.0	1.25	0.91
Gradient $(r_0 - r_{10})$		+0.40	+0.66

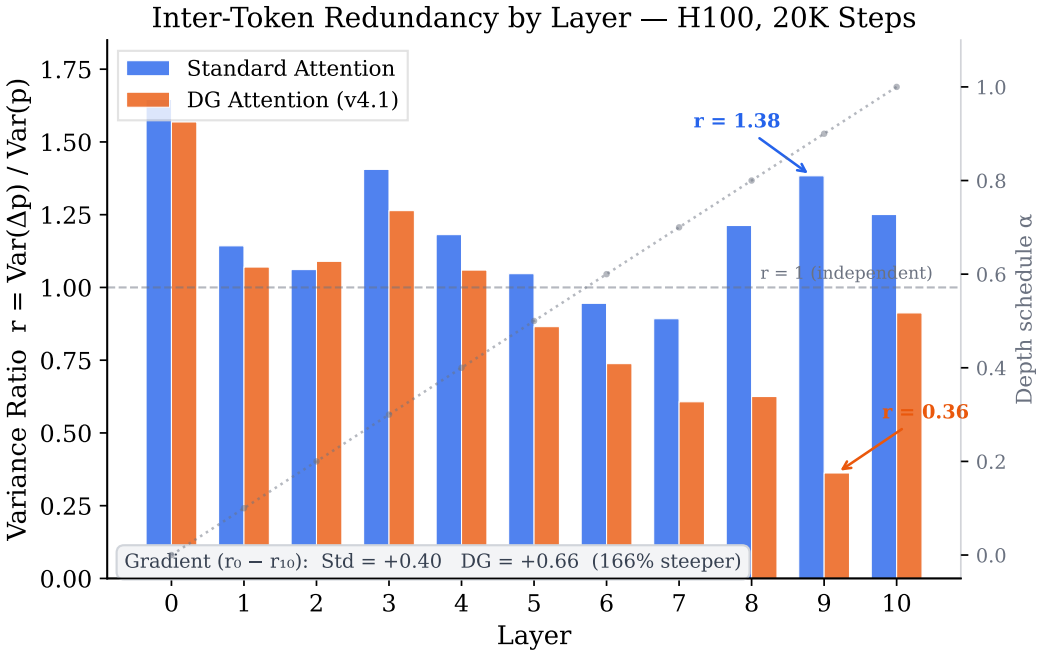

Figure 4: Per-layer variance ratio on the matched H100 runs at 20K steps. Values below $r = 1$ (dashed line) indicate adjacent tokens are *more similar* than independent, i.e., the layer has developed inter-token redundancy. DG amplifies this redundancy gradient: at layer 9, DG achieves $r = 0.36$ while standard attention *reverses* the trend to $r = 1.38$. The dotted gray line shows the depth schedule α_ℓ.

Four findings emerge (combining small-batch and large-batch measurements):

1. Standard attention develops depth-varying inter-token redundancy. At random initialization, $r_\ell \approx 1.0$ for all layers, meaning adjacent tokens' projections are independent. After 2000 steps of training, a strong depth gradient emerges across all seeds: the mean r_ℓ decreases

from 1.45 (shallow) to 0.72 (layer 10). The deepest layer consistently develops the most inter-token redundancy, with $r_{10} < 1.0$ indicating that adjacent-token differences carry less variance than raw projections. This gradient is learned, not architectural, and is robust across seeds.

2. DG amplifies the redundancy gradient at both small and large batch. At 393K batch tokens (2,000 steps, 3 seeds), the DG mean gradient ($r_0 - r_{10} = +0.99$) is steeper than standard's ($+0.73$), with DG producing lower r_ℓ values at every layer. The matched H100 comparison (786K batch, 20,000 steps, Table 8) confirms this pattern at scale: DG's gradient is $+0.66$ versus standard's $+0.40$. Notably, DG achieves $r_9 = 0.36$ (the lowest value at any layer in any condition) while standard's deep layers *reverse* the trend ($r_9 = 1.38$, $r_{10} = 1.25$). This means DG's depth schedule successfully forces deep layers to operate on low-redundancy differential signals, even though the aggregate BPB benefit seen at small batch vanishes at large batch. An earlier preliminary measurement on a 786K-batch compiled model at 2,000 steps showed DG producing a flat profile ($r_\ell \approx 0.91$), suggesting the steeper gradient at 20,000 steps is a long-training phenomenon.

3. The redundancy profile reflects the U-Net architecture. Across both standard and DG models and all seeds, the r_ℓ profile is non-monotonic: it dips near layer 5 (the encoder-decoder boundary where skip connections inject shallow features) before partially recovering. The deepest layer (layer 10), farthest from any skip injection, consistently develops the strongest redundancy. This suggests that the optimal depth schedule should account for skip-connection topology, committing most strongly to differential encoding at the deepest layers and at the skip boundaries where correlation is disrupted.

Limitations. The current results establish that (1) differential payloads improve BPB at small batch sizes and achieve parity at large batch, (2) the depth schedule is necessary (constant-α hurts), (3) DG reduces artifact size and VRAM while maintaining quality, and (4) DG induces a steeper redundancy gradient than standard attention at both small and large batch. They do *not* yet establish:

- Whether the BPB advantage at small batch re-emerges at longer training horizons on H100 (e.g., 50,000+ steps).
- Whether the freed artifact capacity (6% smaller) translates to measurable BPB gains when reinvested in additional layers or width.
- Whether the improvement generalizes beyond 16 MB to larger model scales.
- How differential payloads interact with complementary techniques (SwiGLU, TTT, etc.).

6.2 Resolved and Remaining Experiments

Resolved. Two of the previously planned experiments have been completed:

- **Fixed schedule + large batch on H100** (Section 5.3): DG v4.1 achieves BPB parity with standard attention at 786K batch tokens over 20,000 steps. The fixed schedule avoids gate collapse but does not recover the small-batch BPB advantage.
- **Matched large-batch variance ratio** (Table 8): DG amplifies the redundancy gradient at large batch ($+0.66$ vs $+0.40$), confirming that differential payloads structurally reorganize value representations even when BPB is unchanged.

Remaining. The following experiments would further characterize DG Attention:

1. **Capacity reinvestment**: Use the ∼1 MB freed by DG's smaller artifact to add an extra layer or widen the MLP, then compare against the standard model at the same 16 MB budget. This tests whether DG's efficiency advantage translates to a concrete BPB win when the saved capacity is exploited.

2. **Model scale**: Run at 32 MB and 64 MB artifact sizes. The question is whether the efficiency advantage (smaller artifact at BPB parity) persists, grows, or shrinks at larger scales.

3. **U-shaped schedule**: The H100 variance ratio profile (Table 8) shows standard attention reversing the redundancy trend in deep layers ($r_9 = 1.38$), while DG's deepest layers achieve the strongest compression ($r_9 = 0.36$). A schedule shaped by skip-connection topology rather than a linear ramp may better match the non-monotonic redundancy structure.

4. **Isolating gate collapse**: Run learned-β DG v2.1 at two batch sizes on the same hardware to determine whether batch size alone drives gate collapse, or whether training duration also plays a role.

7 Conclusion

The central finding of this work is a negative result: replacing value projections with differential payloads does not improve language modeling quality at practical training scales. Despite a well-motivated hypothesis and promising small-batch results, matched H100 experiments show DG Attention converging to effectively the same BPB as standard attention (1.155 vs. 1.152 post-quantization), with the small-batch advantage vanishing entirely at larger batch sizes.

DG Attention does produce measurable structural changes: 33% less VRAM, a 6% smaller artifact, and a 66% steeper inter-token redundancy gradient in the value stream (+0.66 vs +0.40). But these differences do not translate to better modeling. The variance ratio measurements are perhaps the most interesting aspect: DG reorganizes how the model represents information across depth, achieving $r_\ell = 0.36$ at its deepest layers versus standard attention's $r_\ell = 1.38$, yet both models reach the same loss. This suggests that the redundancy in standard attention's value stream is not actually wasted capacity — the model can learn around it just as effectively.

The design trajectory itself may be the most useful contribution. The gate-collapse phenomenon (Section 5.4), where learned per-layer gates revert to conservative behavior at large batch sizes, is likely relevant to other gated or mixture-of-experts architectures. The ablations confirming that constant-α variants actively hurt performance underscore that naive application of differential encoding is harmful: only the depth-scheduled variant achieves parity, and even then only parity.

I report this as an honest negative result in the spirit of the Parameter Golf competition's encouragement of "interesting negative results." The hypothesis was reasonable, the implementation was clean, and the answer was no.

References

Kevin Clark, Urvashi Khandelwal, Omer Levy, and Christopher D. Manning. What does BERT look at? An analysis of BERT's attention. In *Proceedings of the 2019 ACL Workshop BlackboxNLP*, 2019.

Jan Kautz, Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, et al. Gated delta networks: Improving mamba2 with delta rule. In *ICLR*, 2025.

Ioanna Lygizou, Máté Farsang, and Radu Grosu. Differential gated self-attention. In *NeurIPS*, 2025.

Catherine Olsson, Nelson Elhage, Neel Nanda, et al. In-context learning and induction heads. *Transformer Circuits Thread*, 2022.

OpenAI. Parameter golf: Compressed language modeling challenge. https://github.com/openai/parameter-golf, 2025.

Guilherme Penedo, Quentin Malartic, Daniel Hesslow, et al. The FineWeb datasets: Decanting the web for the finest text data at scale. *arXiv preprint arXiv:2406.17557*, 2024.

Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. Attention is all you need. In *NeurIPS*, 2017.

Mingyu Xu, Wei Cheng, Bingning Wang, et al. KV shifting attention enhances language modeling. *arXiv preprint arXiv:2411.19574*, 2024.

Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. In *NeurIPS*, 2024.

Tianzhu Ye, Li Dong, Yuqing Xia, Yutao Sun, Yi Zhu, Gao Huang, and Furu Wei. Differential transformer. *arXiv preprint arXiv:2410.05258*, 2024.

Amir Zandieh, Majid Daliri, Majid Hadian, and Vahab Mirrokni. TurboQuant: Online vector quantization with near-optimal distortion rate. In *ICLR*, 2026. *arXiv preprint arXiv:2504.19874*, 2025.

Insu Han, Praneeth Kacham, Amin Karbasi, Vahab Mirrokni, and Amir Zandieh. PolarQuant: Quantizing KV caches with polar transformation. In *AISTATS*, 2026. *arXiv preprint arXiv:2502.02617*, 2025.